

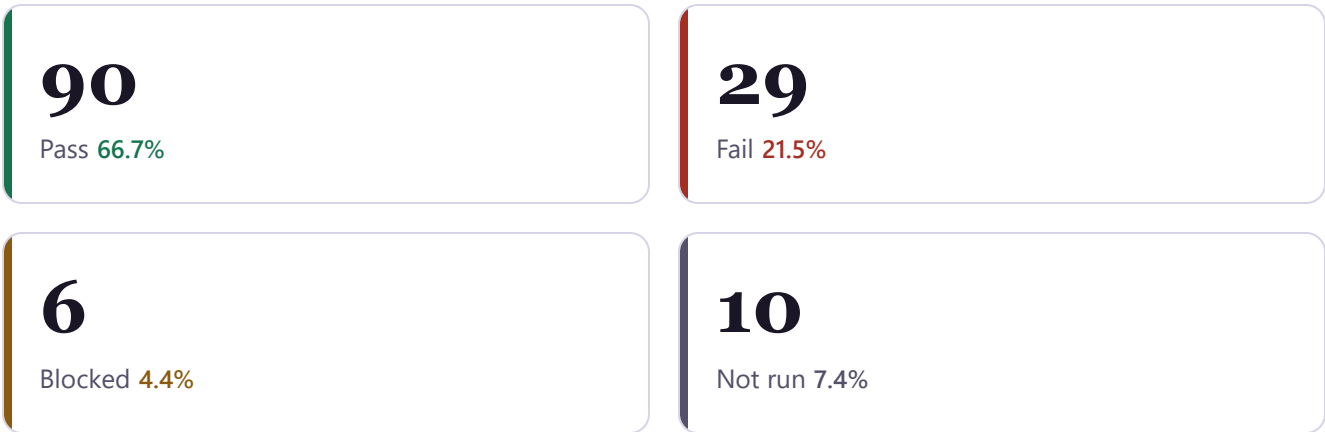
Howler Quality Assurance Audit

A full-surface QA pass across authentication, moderation, messaging, rate limiting, accessibility, and security — executed with real browser automation and live API calls against the production deployment.

- 135 test cases
- 19 categories
- Environment: **Production** (howler-teal.vercel.app)
- Executed 2026-07-18

Results at a glance

Of 135 cases, 119 were actually executed; 16 are Blocked or Not Run for documented reasons (a second test account, out-of-scope test imagery, or infrastructure access this engagement didn't have — see the appendix).



Of the **119** cases actually executed, **75.6%** passed. Two additional defects (**NEW-01**, **NEW-02**) were discovered during testing and are included in the totals above.

Methodology

This was a real-environment audit, not a synthetic test suite run against mocks — every result below reflects an actual request against production.

Real browser automation

Playwright drove actual Chromium, Firefox, and WebKit sessions through the real UI — clicks, form fills, uploads — not simulated events.

Live API validation

Every endpoint was also called directly with an authenticated session to check status codes, payloads, and error handling beneath the UI layer.

Resilience & adversarial cases

Network throttling, forced offline states, delayed/failed responses, tampered cookies, and malformed payloads were used to probe failure paths, not just happy paths.

Guardrails on real users

Every action visible to a real person, billed against a real API, or targeting another user's content (new DMs, reports, image analysis) was confirmed case-by-case before it ran, and test content was cleaned up immediately after.

Explicitly out of scope this round: five AI Image Analysis cases (IMG-02–06) require sourcing real adult or violent test imagery, which was not done under any circumstance; two Text Filtration cases were skipped by scope decision; the remaining Blocked/Not Run items need a second test account, a non-UW login, or Supabase dashboard access not available in this engagement.

Priority findings

Nine issues worth discussing first, ranked by priority. The full list of 29 failures and 6 blocked cases is in the appendix below.

CRITICAL

TXT-02

Comment profanity filter has a word-specific bypass

The word "shit" passes through the comment filter untouched — reproduced twice — while the identical word is correctly blocked in posts, and a different profane word ("fuck") is correctly blocked in comments too. Likely a context-scoring quirk on short text rather than a total failure, but it's a live, reproducible gap in a shipped moderation feature.

CRITICAL

MSG-03

Sending a message has no membership check

GET /api/messages verifies the caller is a conversation participant before returning data; POST /api/messages does not — confirmed by source review. Only a foreign-key constraint stood between this test and an unauthorized insert. Given a real conversation ID, any authenticated user could likely message into a conversation they're not part of.

CRITICAL

SEC-06

Internal database errors reach the client

No credentials or secrets leak, but unhandled exceptions across posts, comments, and other routes return raw Postgres error text — column names, constraint names, type errors — directly in the response body. An internal-schema disclosure worth closing before wider launch.

HIGH

SEC-08

Session cookie is readable by JavaScript

The auth cookie carries SameSite=Lax (a reasonable CSRF baseline) but httpOnly:false and secure:false. No XSS vector exists today (see SEC-01), but if one is ever introduced the session is trivially stealable — this is in fact exactly how test sessions were captured for this audit.

HIGH

SEC-07

A corrupted session cookie crashes the server

Replaying a tampered auth cookie against GET /api/auth returns an unhandled 500 crash instead of a clean 401. Any malformed cookie — tampering, corruption, or a buggy client — takes the request down rather than degrading to "unauthenticated."

HIGH

IMG-07

IMG-09

Bad image uploads crash the post route — and still cost money

A renamed non-image file or a truncated image both crash posts/route.js with an unhandled 500, because Vision returns a null response that the code reads without a null check. Worse: the request still bills a Vision API call and consumes an upload-quota slot before it fails — a bad actor could burn through both budgets with garbage files.

HIGH

A11Y-01

A11Y-02

Core interactions are unreachable by keyboard or screen reader

Like, comment, report, and delete controls are plain <div>s with onClick handlers — no tabindex, no role, no keyboard handler — so they can't be reached with Tab or activated with Enter/Space. The comments modal also lacks role="dialog" and aria-modal, so screen readers won't announce it.

MEDIUM

ERR-04

/api/reports has no error handling at all

Malformed JSON crashes it with a completely empty 500 — no error message whatsoever. Separately, an empty payload is accepted as a valid report, creating a row with a null post and null reason. Of every input-handling gap found this session, this route needs the most immediate attention.

MEDIUM

RT-03

The main feed has no realtime updates

A post created in a second session never appeared in an already-open feed without a manual refresh — PostFeed.js has no Supabase realtime subscription at all. Only direct messages actually have realtime, which contradicts the README's claim of "live updates for interactions and posts."

Also discovered during testing (not in the original plan):

- NEW-01 — GET /api/posts 500s for a fully unauthenticated caller (a null user ID fails a uuid cast in Postgres). Not reachable through normal navigation today, but a real gap in a public API route.
- NEW-02 — Deleting a post never removes its image from Storage; the file stays publicly reachable indefinitely, accumulating cost and outliving the reason it was deleted.

Category breakdown

All 19 categories, sorted by failure count — where the problems concentrate, not just where the volume is.



Full results

All 135 test cases, grouped by category. Search or filter by status — matching sections expand automatically.

135 of 135 test cases shown

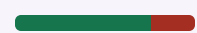
Accessibility				6 / 6
ID	TEST CASE	RESULT	PRIORITY	STATUS
A11Y-01	Keyboard-only navigation through the feed	Like/comment/report/delete controls are plain divs with onClick — no tabindex, no role, unreachable by keyboard.	High	Fail
A11Y-02	Screen reader announces modals and key content	Comments modal has no role="dialog" or aria-modal — won't be announced as a dialog.	High	Fail
A11Y-03	Color contrast meets WCAG AA	axe-core found "serious" contrast violations: 16 elements on the feed, 18 inside the comments modal.	Medium	Fail
A11Y-04	Escape key and focus return close modals	Comments modal does not close on Escape (ReportModal does, per source).	Medium	Fail
A11Y-05	Form validation errors are announced accessibly	The category-required error is plain text with no aria-live region — screen reader users aren't notified.	Medium	Fail
A11Y-06	Images have appropriate alt text	Zero image-alt violations found by axe-core.	Low	Pass

Security				9 / 9
ID	TEST CASE	RESULT	PRIORITY	STATUS
SEC-01	XSS via post content	React's default escaping held — an onerror payload never executed and never rendered as raw HTML.	Critical	Pass

ID	TEST CASE	RESULT	PRIORITY	STATUS
SEC-02	SQL/NoSQL injection via query params	Search/profile safely parameterized; but a non-UUID postId on /api/comments leaks a raw Postgres error via 500.	High	Fail
SEC-03	IDOR via guessed/incremented IDs	No successful unauthorized access found across posts/comments/DMs (consolidates POST-06, MSG-02, MSG-03 findings).	Critical	Pass
SEC-04	Rate limiter can't be bypassed via header spoofing	A deliberately sub-threshold batch wasn't large enough to prove or disprove a bypass — avoided further load on production.	Medium	Blocked
SEC-05	File upload blocks path traversal	A malicious filename was ignored entirely — the storage path is built server-side from the user id and timestamp.	Medium	Pass
SEC-06	Secrets/internals never leak in API responses	No credentials leaked, but several routes return raw Postgres error text directly to the client.	Critical	Fail
SEC-07	Tampered session cookie is rejected cleanly	A corrupted cookie crashes the server with an unhandled 500 instead of a clean 401.	High	Fail
SEC-08	CSRF protection on state-changing requests	SameSite=Lax gives a baseline, but the session cookie is httpOnly:false and secure:false — readable via JavaScript.	High	Fail
SEC-09	Reporter identity isn't exposed to the reported user	report_creator_id never appears in the feed response.	High	Pass

Posts

12 / 12



ID	TEST CASE	RESULT	PRIORITY	STATUS
POST-01	Create a valid text-only post	Created via the real UI flow; appeared correctly in the live feed.	Critical	Pass
POST-02	Post without a category	Succeeded with a null category_id — the "select a category" rule is client-side only, not enforced by the server.	Medium	Fail
POST-03	Feed pagination returns non-overlapping pages	No ID overlap between pages (small current dataset limits deep-pagination coverage).	High	Pass
POST-04	Filter feed by category	Returned only posts matching the requested category.	Medium	Pass
POST-05	User can delete their own post	Deleted via the real UI trigger; confirmed gone from the feed.	High	Pass
POST-06	User cannot delete another user's post	Returned success:true but silently matched zero rows — not actually deleted. Functionally safe, but the misleading response is a UX/API concern.	Critical	Pass
POST-07	Deleting a non-existent post ID	Graceful no-op, success:true, no error.	Low	Pass
POST-08	Deleting a post with no id parameter	Correctly returned 400 'Missing post ID'.	Medium	Pass
POST-09	Like/comment counts on feed items are accurate	Counts matched exactly after a controlled like + 2 comments.	Medium	Pass
POST-10	user_has_liked reflects only the requester's own like	Correct — reflected only the current user's like state.	Medium	Pass
NEW-01	GET /api/posts 500s for a fully unauthenticated caller	A null user id serializes as the string "null" and fails a uuid cast in Postgres. Not reachable through normal navigation, but a real gap in a public route.	Medium	Fail
NEW-02	Deleted posts don't clean up their Storage image	Confirmed: an image stayed publicly reachable (200 OK) minutes after its post was	Medium	Fail

ID	TEST CASE	RESULT	PRIORITY	STATUS
		deleted — delete_post never calls storage.remove().		

Error Handling / Edge Cases

6 / 6 

ID	TEST CASE	RESULT	PRIORITY	STATUS
ERR-01	5xx errors show a friendly message, not raw internals	Consolidates several routes that return raw Postgres errors, an empty crash body, or null-reference text directly to the client.	Medium	Fail
ERR-02	Offline handling while posting	Submitting while offline leaves the compose modal stuck on "Posting..." forever — no try/catch around the fetch.	Medium	Fail
ERR-03	Empty states render correctly	A zero-result search returns clean empty arrays for the UI to render an empty state from.	Low	Pass
ERR-04	Malformed JSON body is rejected cleanly	/api/comments leaks a raw parser error via 500; /api/reports has no try/catch at all and crashes with a completely empty 500 body.	Medium	Fail
ERR-05	Back/forward navigation preserves state	URL correctly returned to the prior search state.	Low	Pass
ERR-06	Concurrent delete requests on the same post	Both concurrent deletes returned success:true harmlessly — consistent no-op behavior, not a race condition.	Low	Pass

AI Image Analysis

10 / 10 

ID	TEST CASE	RESULT	PRIORITY	STATUS
IMG-01	Safe image is approved and posted	Benign synthetic image passed SafeSearch and posted normally.	Critical	Pass
IMG-02	VERY_LIKELY adult image is rejected	Not run — requires sourcing real adult-content test imagery,	Critical	Not run

ID	TEST CASE	RESULT	PRIORITY	STATUS
		out of scope for this engagement.		
IMG-03	POSSIBLE adult-content image is still rejected	Not run, same imagery constraint as IMG-02.	High	Not run
IMG-04	LIKELY/VERY_LIKELY violent image is rejected	Not run, same imagery constraint as IMG-02.	Critical	Not run
IMG-05	POSSIBLE-violence-only image is allowed through	Not run, same imagery constraint as IMG-02.	Medium	Not run
IMG-06	Vision's 'racy' field currently isn't checked	Not run; code review shows detections.racy is never read in posts/route.js.	Medium	Not run
IMG-07	Non-image file renamed to .jpg	Unhandled 500 — Vision returns null for non-image data and the route crashes reading .adult. Still billed the Vision call and burned an upload-quota slot.	High	Fail
IMG-08	Very large image (20MB+)	Rejected at the platform layer (413) before reaching app code — the app has no explicit size cap of its own.	Medium	Pass
IMG-09	Corrupted / truncated image file	Same crash as IMG-07 — null safeSearchAnnotation, unhandled 500.	Medium	Fail
IMG-10	Text-only post skips the Vision check	Confirmed across dozens of text-only posts — no Vision call, no quota consumed.	Low	Pass

Comments			8 / 8	
ID	TEST CASE	RESULT	PRIORITY	STATUS
COM-01	Add a top-level comment	Created via the real UI with correct joined profile data.	High	Pass
COM-02	Add a nested reply	Reply correctly persisted with its parent_id.	High	Pass

ID	TEST CASE	RESULT	PRIORITY	STATUS
COM-03	Comments are returned in correct order	Returned in strict ascending created_at order.	Medium	Pass
COM-04	GET /api/comments with a malformed postId	A non-UUID postId leaks a raw Postgres error via an unhandled 500 (the missing-param case alone is handled cleanly).	Low	Fail
COM-05	User can delete their own comment	Deleted via the real UI; confirmed removed.	High	Pass
COM-06	User cannot delete another user's comment	Not run — no other user's comment was available this session; POST-06 confirms the identical ownership-check pattern elsewhere.	Critical	Not run
COM-07	Comment on a non-existent post_id	Unhandled 500 with a raw foreign-key constraint error instead of a clean 4xx.	Low	Fail
COM-08	Deep (3-level) reply chains thread correctly	Each reply correctly attributed to its direct parent, no breakage.	Medium	Pass

Reports			5 / 5	
ID	TEST CASE	RESULT	PRIORITY	STATUS
RPT-01	Submit a report with a valid reason	Succeeded via direct API call — the report icon only renders for other users' posts, so self-reporting has no UI path (not a security issue).	High	Pass
RPT-02	Report with no reason selected	Server accepted it anyway (reason stored as null) — the 'select a reason' rule is client-side only.	Medium	Fail
RPT-03	Reporting while logged out	Correctly returned 401.	High	Pass
RPT-04	Same post reported multiple times	No uniqueness constraint — the same user can report the same post repeatedly.	Low	Fail

ID	TEST CASE	RESULT	PRIORITY	STATUS
RPT-05	Reported post isn't auto-hidden	Stayed visible after reporting, consistent with a manual-review moderation workflow.	Low	Pass

Realtime & Optimistic UI			5 / 5	
ID	TEST CASE	RESULT	PRIORITY	STATUS
RT-01	Liking updates the UI instantly	Count updated before the network request could have completed.	Medium	Pass
RT-02	Failed like rolls back correctly	Optimistic count correctly reverted after a forced network failure.	High	Pass
RT-03	New posts appear live for other users	Did not appear without a refresh — PostFeed has no realtime subscription at all, contradicting the README's "live updates" claim.	Medium	Fail
RT-04	New posts appear optimistically before the server responds	Not optimistic — the post only appears after the real server response returns; a "Posting..." state is shown instead.	Medium	Fail
RT-05	Realtime reconnects after a dropped connection	A message sent during a simulated outage arrived within 10s of reconnecting.	Medium	Pass

Text Filtration			10 / 10	
ID	TEST CASE	RESULT	PRIORITY	STATUS
TXT-01	Explicit profanity in a post is rejected	Correctly blocked with 400.	Critical	Pass
TXT-02	Profanity in a comment is rejected	"shit" bypassed the comment filter twice (reproduced), while the same word is blocked in posts and "fuck" is blocked in comments — a word/context-specific gap.	Critical	Fail
TXT-03	Leetspeak obfuscation is still caught	Leetspeak variant of a banned word was correctly rejected.	High	Pass

ID	TEST CASE	RESULT	PRIORITY	STATUS
TXT-04	Case variations are caught	All-caps variant correctly rejected.	Medium	Pass
TXT-05	Legit words with an embedded banned substring aren't flagged	"Scunthorpe" posted successfully — no over-flagging.	High	Pass
TXT-06	Context-aware analysis allows benign use of an ambiguous word	Not run this session.	Medium	Not run
TXT-07	Posts over 500 characters are rejected	501-char post correctly rejected server-side. Note: the UI's own textarea caps at 280, well below the server's real 500-char limit.	High	Pass
TXT-08	Comments over 280 characters are rejected	281-char comment correctly rejected.	High	Pass
TXT-09	Empty / whitespace-only content is rejected	All three cases (blank post, empty comment, null comment) correctly rejected.	High	Pass
TXT-10	DMs aren't run through the profanity filter	Not run — skipped by scope decision; source confirms messages/route.js has no filter check.	Medium	Not run

API Request Runtime			8 / 8	
ID	TEST CASE	RESULT	PRIORITY	STATUS
PERF-01	GET /api/posts response time under load	Couldn't time the real path — unauthenticated calls 500 (see NEW-01); only the error path was timed (~1s).	High	Blocked
PERF-02	GET /api/search response time	Returned well under the 800ms target (small production dataset).	Medium	Pass
PERF-03	POST /api/posts with image (Vision round-trip)	Full Vision + Storage + DB round trip completed in 2.2s for a small test image.	High	Pass

ID	TEST CASE	RESULT	PRIORITY	STATUS
PERF-04	Concurrent identical requests shouldn't duplicate posts	Two simultaneous identical POSTs both succeeded — no server-side idempotency, only a client-side disabled-button guard.	Medium	Fail
PERF-05	Behavior when Vision API is slow/unreachable	Can't block outbound calls to Vision from the client side; not exercised.	High	Not run
PERF-06	Deep-page pagination performance	Page 1 vs. page 20 differed by ~45ms; small dataset doesn't stress-test OFFSET pagination at real scale.	Low	Pass
PERF-07	Realtime subscriptions don't leak connections	No errors/degradation observed across many opened/closed channels, but true verification needs Supabase dashboard access.	Medium	Blocked
PERF-08	API responses set sensible cache headers	Static-ish endpoints (categories, campuses) return max-age=0 — a missed caching opportunity, not a bug.	Low	Pass

Direct Messages			8 / 8	
ID	TEST CASE	RESULT	PRIORITY	STATUS
MSG-01	Send a message in an existing conversation	Created and confirmed in the thread.	High	Pass
MSG-02	Can't read a conversation you're not part of	Correctly returned 403 for a non-member request.	Critical	Pass
MSG-03	Can't send into a conversation you're not part of	The POST handler has no participant check at all (unlike GET) — confirmed by source review; this test's ID was only caught by a foreign-key error, not an access check.	Critical	Fail
MSG-04	Messages over 2000 characters are rejected	Correctly rejected before sending.	Medium	Pass

ID	TEST CASE	RESULT	PRIORITY	STATUS
MSG-05	Empty / whitespace-only message is rejected	Correctly rejected, nothing sent.	Medium	Pass
MSG-06	New message appears live for the other participant	Appeared within the watch window via the DM realtime channel, no refresh needed.	High	Pass
MSG-07	Start a new conversation with another user	New conversation created via get_or_create_dm; confirmed idempotent (repeat call returns the same conversation).	High	Pass
MSG-08	Conversation list only shows the user's own conversations	Spot-checked correctly scoped.	Critical	Pass

Mobile vs Desktop Browser			8 / 8	
ID	TEST CASE	RESULT	PRIORITY	STATUS
RESP-01	Feed layout on mobile viewport	No horizontal overflow; mobile bottom nav renders cleanly.	High	Pass
RESP-02	Feed layout on tablet viewport	No overflow, sensible layout.	Medium	Pass
RESP-03	Desktop layout at wide viewport	Renders correctly at 1920×1080; surfaced an uncatalogued "Trending on Howler" panel.	Medium	Pass
RESP-04	Comments modal usability on mobile	Scrollable and dismissible via the close button, but Escape does not close it (same root cause as A11Y-04).	High	Fail
RESP-05	Image upload flow on mobile	File picker, preview, and submit all worked correctly on a mobile viewport.	Medium	Pass
RESP-06	Messages panel usability on small screens	Clean stacked list→thread navigation with a back button, no layout breakage.	Medium	Pass
RESP-07	Touch interactions work without a mouse	Like/comment taps worked identically to clicks; no hover-only functionality found.	High	Pass

ID	TEST CASE	RESULT	PRIORITY	STATUS
RESP-08	Orientation change doesn't break layout	No overflow after rotating to landscape.	Low	Pass

Search			5 / 5	
ID	TEST CASE	RESULT	PRIORITY	STATUS
SRCH-01	Search posts by partial content	Correct substring matches returned.	High	Pass
SRCH-02	Search profiles by username/name	Correct matches returned.	High	Pass
SRCH-03	Empty search query	Returned empty arrays without querying the DB.	Medium	Pass
SRCH-04	Wildcard/injection-style queries are sanitized	SQLi payload safely neutralized — but a literal '%' query returns a broad "match everything" result since input isn't escaped before entering the ilike pattern.	High	Fail
SRCH-05	Search results exclude sensitive fields	Only id/username/full_name/avatar_url exposed.	Medium	Pass

Likes			4 / 4	
ID	TEST CASE	RESULT	PRIORITY	STATUS
LIKE-01	Like a post for the first time	Count went 0→1 via the real UI tap, confirmed server-side.	High	Pass
LIKE-02	Unlike (toggle off)	Count correctly reverted 1→0.	High	Pass
LIKE-03	Rapid double-tap doesn't desync the count	Two concurrent requests resolved deterministically — one liked, one unliked, net change zero.	Medium	Pass
LIKE-04	Like a non-existent post_id	Unhandled 500 with a raw foreign-key constraint error.	Low	Fail

Authentication			8 / 8	
-----------------------	--	--	-------	------------------------

ID	TEST CASE	RESULT	PRIORITY	STATUS
AUTH-01	Google OAuth login with a UW account	Observed directly: real UW Google login redirected correctly, session cookie set, username auto-generated from email.	Critical	Pass
AUTH-02	Microsoft OAuth login (UW Azure tenant)	Not exercised this session (logged in via Google); JWT shows a linked, previously-used UW Azure identity as indirect evidence.	Critical	Blocked
AUTH-03	Non-@uw.edu Google account should be rejected	Needs a non-UW Google account. LoginButton only sends a client-side hd:'uw.edu' hint — not confirmed enforced server-side.	Critical	Not run
AUTH-04	GET /api/auth with no session	Correctly returned 401 {authenticated:false}.	High	Pass
AUTH-05	Session persists across reload / new tab	Stayed authenticated after reload and in a fresh tab.	High	Pass
AUTH-06	Logout clears session fully	Logout returned session to logged-out state immediately; run last since it invalidates the test session.	Critical	Pass
AUTH-07	Repeat login doesn't duplicate the profile row	Profile stayed consistent across two logins; not directly row-counted in the DB.	High	Pass
AUTH-08	Unauthenticated user can't call mutating routes	All 7 mutating endpoints correctly returned 401 with no side effects.	Critical	Pass

Rate Limiting			8 / 8	
ID	TEST CASE	RESULT	PRIORITY	STATUS
RATE-01	General reads capped at 60/min	Burst of 65 unauthenticated GETs: succeeded until the shared window filled, then correctly 429'd.	High	Pass
RATE-02	Writes capped at 20/min	22-request burst: rejected exactly the requests over the 20/min	High	Pass

ID	TEST CASE	RESULT	PRIORITY	STATUS
		sliding window.		
RATE-03	Rate limit is scoped per-user, not global	Source-confirmed (key is user:{id}); RATE-02's exact counts are indirect behavioral confirmation.	Critical	Pass
RATE-04	Falls back to IP for anonymous requests	Anonymous burst was throttled per-IP with correct headers.	Medium	Pass
RATE-05	Sliding window resets after cooldown	Confirmed via the image-upload limiter's real 3-minute reset; general limiter reset is source-confirmed.	Medium	Pass
RATE-06	Image uploads capped at 3 per 3 minutes	3rd upload succeeded, 4th correctly blocked with 429.	High	Pass
RATE-07	Image upload throttle resets after 3 minutes	3 further uploads succeeded after waiting out the window.	Medium	Pass
RATE-08	429 responses include backoff headers	X-RateLimit-Limit/Remaining/Reset all present and accurate.	Low	Pass

Profile Management			6 / 6	
ID	TEST CASE	RESULT	PRIORITY	STATUS
PROF-01	View own profile page	Correct data returned.	High	Pass
PROF-02	View another user's public profile	Correct profile + posts returned.	High	Pass
PROF-03	Non-existent username	Clean 404, no crash.	Medium	Pass
PROF-04	Edit profile details	Bio update persisted and reverted cleanly.	High	Pass
PROF-05	Username collision handling	Not testable without a second colliding-email account; source shows no uniqueness/suffix logic on the derived username.	Medium	Blocked

ID	TEST CASE	RESULT	PRIORITY	STATUS
PROF-06	My-profile likes list is accurate	Correctly reflected a controlled like/unlike.	Medium	Pass

Cross-Browser Compatibility			5 / 5	
ID	TEST CASE	RESULT	PRIORITY	STATUS
BROW-01	Core flows in Chrome	Feed loads cleanly, zero console errors.	High	Pass
BROW-02	Core flows in Safari (WebKit engine)	Clean in WebKit, though this isn't identical to real Safari's ITP/cookie behavior — a strong signal, not a substitute.	High	Pass
BROW-03	Core flows in Firefox	Clean, aside from a benign Cloudflare cookie-scope console warning from Supabase's storage CDN (not a Howler bug).	Medium	Pass
BROW-04	Core flows in Edge	Playwright has no real Edge engine; rendering is likely fine (Chromium-based) but the Microsoft/Azure SSO interplay wasn't verified.	Medium	Blocked
BROW-05	Graceful degradation on a throttled connection	Shows a proper loading state rather than a blank screen, though the full feed took ~42s under heavy throttling.	Medium	Pass

Categories & Campus			4 / 4	
ID	TEST CASE	RESULT	PRIORITY	STATUS
CAT-01	Category list loads for post creation	11 categories returned in 0.42s. One data-hygiene issue: "Pictures/Art" has a stray leading newline in its stored name (not visibly rendered).	Medium	Pass
CAT-02	Category filter chip filters the feed	Clicking a chip correctly re-fetches and re-renders the filtered feed.	Medium	Pass

ID	TEST CASE	RESULT	PRIORITY	STATUS
CAT-03	Campus list loads correctly	3 campuses returned correctly.	Low	Pass
CAT-04	Profile shows the correct campus name via join	Resolved correctly to "UW Seattle".	Low	Pass

Howler QA Audit — 135 test cases across 19 categories, executed against production on 2026-07-18. Prepared for internal engineering review.